

Name and Surname: \_\_\_\_\_

Student ITS No: \_\_\_\_\_

Qualification: \_\_\_\_\_ Year of Study: \_\_\_\_\_ Semester: \_\_\_\_\_

Assignment due date: \_\_\_\_\_ Date submitted: \_\_\_\_\_

| QUESTION | EXAMINER MARKS | MODERATOR MARKS | REMARKS |
|----------|----------------|-----------------|---------|
|          |                |                 |         |
|          |                |                 |         |
|          |                |                 |         |
|          |                |                 |         |
|          |                |                 |         |
|          |                |                 |         |
|          |                |                 |         |
|          |                |                 |         |

**ASSIGNMENT INSTRUCTIONS**

*Please tick each box to confirm completion.*

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.

Apply Harvard Referencing Style for all citations and references.

**For essay-style assignments, please include the following sections:**

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle.

Use the specified cover page provided.

Include a signed declaration of originality.

**DECLARATION OF ORIGINALITY:**

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: \_\_\_\_\_

Date: \_\_\_\_\_

**FACULTY OF INFORMATION TECHNOLOGY**

**MOBILE APP DEVELOPMENT 700**

# **PRACTICAL ASSIGNMENT**

## **Smart Pantry Manager**

*A Java Android Application that Suggests Recipes Based Strictly on Leftover  
Ingredients to Cut Food Waste*

Portfolio of Evidence: Application, GitHub Repository, Video Demonstration & Written Report

**Total: 100 Marks**

**Individual Assignment**

## 1. Introduction

This assignment forms part of the practical assessment for Mobile App Development 700. Building on the module content covering Android Studio, the Activity lifecycle, layouts, intents, adapters, and persistent data storage, you are required to design, build, document, and demonstrate a fully functional Android application written in Java.

## 2. Assignment Brief

### 2.1 Project: Smart Pantry Manager

For this assignment, you will design and build a single, defined application: the Smart Pantry Manager, written in Java.

**Concept:** the Smart Pantry Manager helps a user reduce food waste by tracking the ingredients they actually have at home (their "pantry") and suggesting recipes they can cook using strictly those leftover ingredients - no shopping trip required, and no recipe suggested unless the user genuinely already has everything it needs.

The core value of the app is this strict-matching rule, and it is treated as the single most important piece of business logic in the assignment (see Section 2.3).

### 2.2 Required Functionality

- Pantry management - the user can add, edit, and delete pantry items (e.g. ingredient name, quantity, unit, and optionally an expiry date).
- A pantry list screen showing all current ingredients, using a RecyclerView/ListView bound to your database.
- A recipe collection - a set of at least 15-20 recipes stored in your database (pre-loaded/seeded on first run), each recipe consisting of a name, a list of required ingredients, and simple preparation steps.
- A "Suggested Recipes" screen that runs the strict-matching logic (Section 2.3) against the user's current pantry and lists only the recipes the user can make right now.

- A recipe detail screen showing the full ingredient list and method for a selected recipe.
- A settings or profile screen (e.g. toggle for expiring-soon alerts, or units preference), satisfying the minimum-screens requirement in Section 3.1.
- Basic feedback when zero recipes match the current pantry (e.g. “No recipes match your pantry yet - add more ingredients”), rather than a blank or broken screen.

### 2.3 The Strict-Matching Rule (Core Logic)

A recipe may only be shown as “suggested” if every single ingredient it requires is currently present in the user's pantry, in at least the required quantity. This is what “strictly” means in the brief, and it is the piece of logic your marker will test directly:

- If a recipe needs 5 ingredients and the user's pantry has 4 of them, that recipe must NOT appear in the suggestions list.
- Partial matches, “almost there” recipes, or recipes requiring one missing ingredient must be excluded from the main suggestions list. (Optional stretch: a separate “Almost There” list showing recipes missing only 1 ingredient may be added for bonus credit - see Section 8 - but it must be clearly separated from the strict suggestions.)
- Matching should be reasonably robust to simple real-world messiness (e.g. unit differences, singular/plural ingredient names) - you do not need a full NLP solution, but a naive exact-string match that breaks on trivial differences (“tomato” vs “tomatoes”) will be marked down.

**Important restriction:** your application must NOT use Google Maps, any mapping SDK, or device location/GPS services. This includes finding “nearby grocery stores” or similar location-based features - the app's scope is strictly the user's own pantry and recipe matching.

## 3. Technical & Functional Requirements

### 3.1 Core Requirements

- Built entirely in Java (not Kotlin) using Android Studio, consistent with the module's IDE and syntax.
- A minimum of four (4) distinct screens/Activities (or Fragments navigated through a single host Activity): the Pantry List, Add/Edit Ingredient, Suggested Recipes, and Recipe Detail screens described in Section 2.2, plus a Settings screen.

- Correct use of Intents to navigate between screens and pass data.
- At least one RecyclerView or ListView with a custom Adapter to display a dynamic list of data from your database.
- A working navigation element (bottom navigation bar, toolbar menu, or navigation drawer).
- Input validation on any data entry form.
- A clear, legible layout suitable for a mobile screen, using LinearLayout, RelativeLayout, ConstraintLayout, or a Grid layout as appropriate.

### 3.2 Database Requirement (choose ONE)

Your application must persist user data using one of the following. All three are acceptable - choose the one you are most comfortable justifying in your report and video.

- **SQLite** - using SQLiteOpenHelper or the Room persistence library, implemented locally on-device, consistent with the database approach covered in the module's persistent data chapter.
- **Firebase** - using Firebase Realtime Database or Cloud Firestore for cloud-based storage and sync.
- **PostgreSQL** - hosted locally or on a free cloud tier (e.g. Supabase, Render, ElephantSQL), accessed from the app via a simple REST API/backend that you also build or configure.

Whichever option you choose, your app must demonstrate full CRUD functionality: a user must be able to Create a new record, Read/view existing records in a list, Update an existing record, and Delete a record - and the data must still be present after the app is closed and reopened (i.e. it must genuinely persist, not just live in memory).

### 3.3 Explicitly Out of Scope

- Google Maps, any third-party mapping SDK, or GPS/location-based features.
- Payment processing or real financial transactions.
- Publishing the app to the Google Play Store (not required for this assignment).

## 4. Version Control Requirement (GitHub)

You must develop your project under Git version control on GitHub from the start of development - not create a repository at the end and upload a single, final commit.

- Create a GitHub repository (PUBLIC) for the project (private repositories must add the lecturer/marker as a collaborator - details will be provided on the submission portal).
- Make a minimum of 10 commits over the course of development, each reflecting a genuine, incremental step (e.g. "Add ContactListActivity layout", "Implement DatabaseHelper CRUD methods", "Fix null pointer on empty list").
- Write clear, descriptive commit messages - avoid single commits like "final" or "update".
- Include a README.md in the repository containing: app name and description, the database option you chose and why, and clear setup/run instructions (see Section 6).

*Note: your GitHub commit history is treated as evidence of your own development process and is checked against your video demonstration (Section 5) and report timeline. A repository with no meaningful history, or with all commits made in a single sitting immediately before the deadline, will be flagged for further investigation.*

## 5. Video Demonstration (5-7 minutes)

You must record a single video, 5 to 7 minutes long, that proves both that your app works and that you understand it. Videos shorter than 5 minutes or longer than 7 minutes will be stopped/assessed only up to the limit. Screen-record using a tool such as OBS Studio, the Android Studio screen recorder, Loom, or your phone's built-in screen recorder (for a physical device demo) etc.

### 5.1 Required Structure

1. **GitHub walkthrough (approx. 1 minute):** open your GitHub repository and scroll through your commit/push history on screen, briefly narrating how the project evolved over time.
2. **Live app demonstration (approx. 2-3 minutes):** run the app (emulator or physical device) and demonstrate every core feature working, including a full Create, Read, Update and Delete cycle on pantry items, and prove the strict-matching rule works correctly - e.g. add/remove one ingredient on screen and show a recipe appear in or disappear from the Suggested Recipes list accordingly. Show the data still present after closing and reopening the app.

3. **Concept explanation (approx. 2-3 minutes):** with your code open, verbally explain - in your own words - three of the following: the Activity (or Fragment) lifecycle in your app, how your chosen database works end-to-end, how your RecyclerView/ListView and Adapter display data, how Intents move between screens in your app, or how your strict-matching algorithm (Section 2.3) decides whether a recipe qualifies as "suggested". Point to the actual lines of code as you talk.
4. **Database justification (approx. 30 seconds-1 minute):** briefly explain why you chose SQLite, Firebase, or PostgreSQL for this app.

## 5.2 Video Requirements

- Your face does not need to be on camera, but your voice narrating throughout is compulsory - a silent screen recording will not be accepted.
- Screen must be clearly legible (record at a minimum of 1080p where possible).
- Compress your video to a reasonable file size (e.g. export as MP4 using H.264 encoding) and include the video file directly inside your submission ZIP (see Section 7) - do not submit only a link to an external hosting site.

## 6. Written Report Requirements

Submit a written report (Word or PDF) containing the following sections, in this order:

1. Cover page - app name, your name, student number, module name, date.
2. Table of contents.
3. Introduction - the problem your app solves and who it is for.
4. System design - a simple screen flow/wireframe diagram showing how your screens connect, and a simple data model/ER diagram of your database table(s)/collection(s).
5. **Screenshots of every output** - an actual screenshot of your running app for every screen and every core function (add, edit, delete, list view, validation error, etc.), each with a short caption explaining what it shows. Do not use placeholder images, stock icons, or design mockups in place of real screenshots of your own running app.
6. Key code snippets - 3 to 5 short, well-chosen snippets (not your entire codebase) with a brief explanation of what each does and why it was written that way.

7. Challenges and solutions - 2 to 3 real problems you encountered and how you resolved them.
8. Conclusion and reflection - what you learned, and what you would improve given more time.
9. Reference list - any tutorials, documentation, or sources consulted, in a consistent referencing style.

## 7. Submission Instructions

Submit a single ZIP file via **Moodle** by the due date, named:

Studentnumber\_Surname\_MobileAppDev700\_Assignment.zip - containing the following:

- Your full Android Studio project source code (the complete project folder - you may exclude the build/ and .gradle/ subfolders to keep the file size manageable).
- Your 5-7 minute video demonstration, compressed (see Section 5.2) and placed directly inside the ZIP file as a video file - do not submit only a link.
- Your written report as a single PDF or Word document, named:  
Studentnumber\_Surname\_MobileAppDev700\_Assignment.docx - this report must include a working link to your GitHub repository (ensure the repository is accessible to the marker).

Do not submit these items separately or as external links (other than the GitHub link inside your report) - the source code, compressed video, and report must all be contained within the one ZIP file.

**File size limit:** the complete submission ZIP file must not exceed 50MB in total. Compress your video appropriately (see Section 5.2) and exclude unnecessary build files from your source code to stay within this limit - submissions over 50MB may be rejected by the submission portal or not accepted for marking.

**Late submissions:** follow the standard institutional late submission policy unless otherwise communicated by your lecturer.

## 8. Marking Rubric

| Criteria                         | Description                                                                                      | Marks |
|----------------------------------|--------------------------------------------------------------------------------------------------|-------|
| App Functionality & Requirements | All required screens work; CRUD operations function correctly; app is stable and free of crashes | 20    |

|                                  |                                                                                                             |     |
|----------------------------------|-------------------------------------------------------------------------------------------------------------|-----|
| Database Implementation          | Correct, working use of SQLite, PostgreSQL or Firebase; data persists between sessions; sensible data model | 15  |
| Code Quality & Java Fundamentals | Clean, readable, commented Java code; correct use of Activities, Layouts, Intents and Adapters              | 25  |
| UI/UX Design                     | Interface is usable, consistent and appropriately laid out for a mobile screen                              | 10  |
| GitHub Version Control           | Regular, meaningful commits showing incremental progress; complete push history; README present             | 10  |
| Video Demonstration              | 5-7 minutes; shows GitHub history, working app, and clear verbal explanation of concepts                    | 10  |
| Written Report                   | Complete, well-structured, includes all required screenshots and sections                                   | 10  |
| TOTAL                            |                                                                                                             | 100 |

## 9. Academic Integrity Declaration

By submitting this assignment, you confirm that:

- This app concept and its implementation are your own original work.
- You are able to explain and defend any part of your submitted code if asked to do so.
- You have not copied another student's app concept, code, report, or video.

Submitting work that is not your own, or work you cannot explain when questioned, will be treated as academic dishonesty under the institution's assessment policy.